

数据仓库与数据挖掘大作业选题思路与建议

1. 作业要求再理解

根据 `大作业要求.txt`、`选题/选题要求.txt` 和 `chap0-about-大作业说明部分.txt`，本次大作业的核心不是单纯实现一个工具函数，而是选择一个明确的时间序列数据挖掘任务，并将对应算法封装为 Apache IoTDB 函数库中的 UDF 算子。

课程说明中给出了四类可选任务：

- 预测任务：预测未来一段窗口内的若干数据点。
- 分类/聚类任务：寻找同质性高、彼此差异明显的时间序列类别或簇。
- 异常检测任务：发现时间序列中的异常点或异常子序列。
- 模式发现任务：发现长时间序列中反复出现的子序列模式，即 motif。

目前计划优先在以下三个方向中选择一个：

- 预测。
- 分类/聚类。
- 异常检测。

因此，下面只围绕这三个方向给出建议。每个方向保留两个可落地选题，便于后续比较实现难度、展示效果和报告可写性。

一个合适的选题应当同时满足：

- 属于时间序列数据挖掘任务之一。
- 算法有明确来源和清楚的输入输出定义。
- 能在 IoTDB UDF 中以 SQL 方式调用。
- 可以设计测试用例和实验结果图。
- 实现复杂度适合课程大作业，不依赖过重的深度学习框架。

选题报告阶段需要重点写清楚：

- 研究背景与问题定义：应用场景、领域数据特点、研究动机、问题目标。
- 算法介绍与实现：核心算法介绍、设计思路、IoTDB UDF 系统实现方案。

后续完整大作业还需要提交：

- 大作业展示 PPT。
- 实现代码和说明文档。
- 测试用例和实验描述。

2. 参考资料对选题的启发

课程说明中给出的参考资料包括：

- IoTDB UDF 开发文档：用于理解 UDF 生命周期、输入参数、输出类型和注册方式。
- Apache IoTDB `library-udf` 代码：用于参考官方函数库算子的工程结构和实现风格。
- IoTDB 现有函数库算子：用于避免选题和已有函数过度重复。
- Time Series Library Dataset：可用于预测、异常检测等任务。
- UCR Time Series Classification Archive：可用于分类/聚类任务。

从参考优秀作业看，一个可落地的大作业通常包含三层：

- 算法核心层：实现一个清楚的时间序列挖掘算法。
- UDF 封装层：将算法封装为 IoTDB UDF，例如 `UDTFForecast`、`UDTFAnomalyScore` 等。
- 实验展示层：使用真实或公开数据集展示方法适用场景、不适用场景和结果图。

因此，本组选题不宜泛泛地写“时序数据分析”，而应明确到某个算法和某类任务，例如“基于 Holt-Winters 的预测”“基于 DTW 的聚类”“基于 LOF 的异常检测”等。

3. 预测方向建议

建议 1：基于 Holt-Winters 指数平滑的 IoTDB 时间序列预测 UDF

Holt-Winters 指数平滑方法适合具有趋势和季节性的时间序列预测。它通过水平项、趋势项和季节项递推更新，对未来若干时间点进行预测。

可实现 UDF：

- `hw_forecast(series, period, horizon, alpha, beta, gamma)`：输出未来 `horizon` 个点的预测值。
- `hw_fitted(series, period, alpha, beta, gamma)`：输出历史时间点上的拟合值，便于和真实值对比。

优点：

- 算法经典，公式清楚，适合写选题报告。
- 不依赖机器学习框架，Java 实现可控。
- 实验指标标准，可以使用 MAE、MSE、RMSE、MAPE。
- 对电力负荷、温度、访问量等周期性数据比较合适。

风险：

- IoTDB UDF 对“未来时间戳输出”的处理需要设计清楚。
- 参数 `period`、`alpha`、`beta`、`gamma` 对结果影响较大，需要做参数说明。
- 对突变强、规律弱的数据效果可能一般。

适合数据：

- 电力负荷数据。
- 温度、流量、访问量数据。
- Time Series Library Dataset 中的预测类数据。

综合评价：稳妥、规范、实现难度中等，适合作为预测方向的首选。

建议 2：基于滑动窗口线性回归的 IoTDB 短期趋势预测 UDF

该方法用最近一段滑动窗口内的数据拟合线性趋势，再外推未来若干点。相比 Holt-Winters，它不强调季节性，更适合做短期趋势预测和局部变化预测。

可实现 UDF：

- `linear_forecast(series, window, horizon)`：基于最近 `window` 个点预测未来 `horizon` 个点。
- `rolling_linear_fit(series, window)`：输出每个窗口对应的拟合值或趋势斜率。

优点：

- 实现难度低，算法容易解释。
- UDF 输入输出设计简单。
- 可以额外输出趋势斜率，让实验结果更直观。
- 适合与移动平均预测、朴素预测方法做对比。

风险：

- 算法深度不如 Holt-Winters。
- 只适合短期趋势预测，不能很好处理复杂季节性。
- 如果只实现线性外推，报告中需要通过对比实验体现价值。

适合数据：

- 传感器温度短期趋势。
- 设备负载、CPU 使用率、流量数据。
- 人工构造的上升、下降、平稳趋势序列。

综合评价：实现最简单，但选题亮点偏弱。适合作为低风险备选。

4. 分类/聚类方向建议

建议 1：基于 DTW-KMeans 的 IoTDB 时间序列片段聚类 UDF

DTW 可以衡量两段时间序列在时间轴拉伸或压缩情况下的相似性，KMeans 可用于将多个时间序列片段划分为若干类别。该方向可以将一条长序列切分为多个窗口片段，再使用 DTW 距离进行聚类。

可实现 UDF：

- `dtw_cluster(series, window, k, maxIter)`：输出每个窗口子序列所属簇编号。
- `dtw_distance(series, query, window)`：输出每个窗口与查询模式的 DTW 距离。

优点：

- DTW 是时间序列领域经典距离度量，报告可写性强。
- 聚类结果可以通过不同颜色标出，展示效果直观。
- 可以用 UCR 数据集或自构造多模式数据进行实验。
- 能体现“普通欧氏距离不适合错位时间序列”的问题。

风险：

- DTW-KMeans 计算量较大，需要控制窗口数量和窗口长度。
- 多条序列输入在 IoTDB UDF 中处理较复杂；建议第一版只做单条长序列的窗口聚类。
- 聚类评价需要设计好，可以使用轮廓系数、可视化或与已知标签对比。

适合数据：

- UCR Time Series Classification Archive。
- 设备不同运行状态片段。
- 周期性负载片段。

综合评价：选题质量较高，但实现和实验组织难度也较高。

建议 2：基于 PAA 特征与 KMeans 的 IoTDB 时间序列聚类 UDF

PAA (Piecewise Aggregate Approximation) 先将每个时间序列窗口分段求均值，把原始序列降维为固定长度特征向量，再用 KMeans 聚类。相比 DTW-KMeans，它牺牲一部分弹性匹配能力，但实现更简单、计算更快。

可实现 UDF：

- `paa_cluster(series, window, segments, k, maxIter)`：输出每个窗口片段的簇编号。
- `paa_feature(series, window, segments)`：输出窗口的 PAA 特征摘要。

优点：

- 算法链路清楚：滑动窗口、PAA 降维、KMeans 聚类。
- 计算量比 DTW-KMeans 小，更适合课程项目实施。
- 参数较少，实验可控。
- 可视化展示方便，可以画出不同簇的典型窗口形状。

风险：

- 相比 DTW，算法特色略弱。
- 对时间轴错位较明显的模式不够鲁棒。
- UDF 如果要输出多维 PAA 特征，需要设计字符串输出或拆分为多个函数。

适合数据：

- 周期性设备运行片段。
- 电力负荷日内曲线。
- UCR 中长度固定的单变量时间序列。

综合评价：分类/聚类方向中更稳妥的方案，适合在实现风险和展示效果之间折中。

5. 异常检测方向建议

建议 1：基于滑动窗口统计量与 CUSUM 的 IoTDB 时间序列变化点检测 UDF

变化点检测用于识别时间序列统计性质发生变化的位置，例如均值突变、方差突变、趋势切换等。可以使用左右滑动窗口的均值差、方差差、趋势差异和 CUSUM 累积统计量计算变化分数。

可实现 UDF：

- `change_score(series, leftWindow, rightWindow, method)`：输出每个时间点的变化分数。
- `change_point(series, leftWindow, rightWindow, threshold)`：输出变化点标签。

优点：

- 实现相对简单，落地风险低。
- 图形展示清楚，变化点可以直接标在原始曲线上。
- 适合工业设备工况切换、传感器漂移、流量突增等场景。
- 可以从均值差扩展到方差差、趋势差异和 CUSUM，深度可控。

风险：

- 如果只做均值差，算法深度可能不足。
- 阈值选择需要解释，可以增加分位数阈值或标准化分数。
- 对周期性强的数据容易把正常周期切换误判为变化点。

适合数据：

- 设备状态切换数据。
- 流量、温度、电压等监控序列。
- NAB、AIOps KPI、UCR Time Series Anomaly Archive。

综合评价：最稳妥、展示效果好，是异常检测方向的首选。

建议 2：基于 LOF 的 IoTDB 时间序列异常检测 UDF

LOF (Local Outlier Factor) 通过比较样本与邻域样本的局部密度差异识别离群点。用于时间序列时，可以先将滑动窗口转换为特征向量，例如均值、方差、最大值、最小值、斜率、变化幅度等，再计算每个窗口的 LOF 异常分数。

可实现 UDF：

- `lof_anomaly(series, window, k)`：输出每个窗口的 LOF 异常分数。
- `lof_label(series, window, k, threshold)`：输出异常标签。

优点：

- 算法比简单阈值法更有数据挖掘感。
- 输出异常分数，结果图直观。
- 可以检测形态异常窗口，而不只是单点异常。

- 可与 Z-Score、移动平均阈值法进行对比。

风险：

- 朴素 LOF 复杂度较高，需要控制数据规模。
- 需要解释从原始序列到窗口特征向量的转换。
- 参数 `window`、`k`、`threshold` 对结果影响明显。

适合数据：

- 服务器监控指标。
- 工业设备传感器数据。
- NAB、AIOps KPI、UCR Time Series Anomaly Archive。

综合评价：效果展示好，算法深度较高，但实现复杂度高于变化点检测。

6. 数据与测试数据可行性评估

6.1 Holt-Winters 指数平滑预测

公开数据可用性：高。

适合数据：

- Monash Time Series Forecasting Repository：包含 M4、旅游、智能电表、交通、天气、太阳能等多种预测数据集，适合做标准预测实验。
- UCI Individual Household Electric Power Consumption：家庭电力消耗数据，适合做电力负荷预测。
- UCI Traffic Flow Forecasting：交通流量预测数据，适合做短期流量预测实验。

构造测试数据难度：低。

可以构造如下序列：

```
x(t) = 100 + 0.2t + 10 * sin(2πt / 24) + noise
```

该序列同时包含趋势、季节性和噪声，非常适合验证 Holt-Winters 是否能拟合周期并预测未来点。

结论：数据最容易找，评价指标也标准。适合做正式实验。

6.2 滑动窗口线性回归短期预测

公开数据可用性：高。

适合数据：

- 与 Holt-Winters 相同，可以使用电力、交通、温度、CPU 使用率等连续时间序列。
- 若希望降低难度，可以只截取公开数据中的一小段局部趋势区间做短期预测。

构造测试数据难度：极低。

可以构造如下序列：

```
x(t) = 2t + noise
x(t) = -0.5t + 100 + noise
x(t) = piecewise_linear(t) + noise
```

这类数据可以清楚验证线性回归能否识别上升、下降和平稳趋势。

结论：数据和测试都最容易，但算法本身偏简单，正式选题亮点不足。

6.3 DTW-KMeans 时间序列片段聚类

公开数据可用性：中到高。

适合数据：

- UCR Time Series Classification Archive：包含大量带类别标签的单变量时间序列，可用来验证聚类结果是否能大致对应真实类别。
- 单条长序列也可以人为切成多个窗口，再对窗口做无监督聚类。

构造测试数据难度：中。

可以构造三类不同形状的窗口片段：

```
class A: sin wave
class B: square wave
class C: spike pattern
```

再把这些片段随机拼接成一条长序列，并加入少量时间偏移和噪声。DTW 的优势正好体现在“形状相似但时间位置略有错位”的片段上。

结论：有合适公开数据，但 IoTDB UDF 内做 DTW-KMeans 的工程和计算压力较大。测试数据可以构造，但比预测和变化点检测略麻烦。

6.4 PAA 特征 + KMeans 聚类

公开数据可用性：中到高。

适合数据：

- UCR Time Series Classification Archive：选择长度固定、类别数较少的数据集，转为窗口样本后聚类。
- 电力负荷日曲线：把每天 24 小时或 48 个半小时点作为一个窗口，聚类出工作日、周末、高峰日等模式。
- 交通流量日曲线：按天切片后聚类出不同交通模式。

构造测试数据难度：低到中。

可以构造几类固定长度窗口：

```
class A: 早高峰 + 晚高峰
class B: 单峰
class C: 平稳低负载
```

PAA 会把每个窗口压缩成低维特征，KMeans 应能把这些形状明显不同的窗口分到不同簇。

结论：比 DTW-KMeans 更容易准备数据，也更容易控制规模。适合做稳妥的聚类方向。

6.5 滑动窗口统计量 + CUSUM 变化点检测

公开数据可用性：高。

适合数据：

- Numenta Anomaly Benchmark (NAB)：包含真实和人工时间序列异常数据，适合做监控指标异常检测。
- AIOps KPI Anomaly Detection：适合运维监控类 KPI 异常检测。
- UCR Time Series Anomaly Archive：适合做学术异常检测补充实验。
- UCI Traffic Flow Forecasting 或交通流量数据：适合观察流量状态切换。

构造测试数据难度：极低。

可以构造如下序列：

```
1 到 100 点：均值为 0，方差较小
101 到 200 点：均值为 5，方差较小
201 到 300 点：均值为 5，方差变大
301 到 400 点：出现上升趋势
```

这样可以分别验证均值突变、方差突变和趋势变化。变化点位置已知，便于做正确性测试。

结论：公开数据和人工数据都非常方便，是六个选题中数据准备最稳的方向。

6.6 LOF 窗口异常检测

公开数据可用性：高。

适合数据：

- NAB：适合检测服务器指标、流量、点击量等异常。
- AIOps KPI Anomaly Detection：适合 KPI 指标异常窗口检测。
- UCR Time Series Anomaly Archive：适合选择长度适中的单变量异常检测序列。

构造测试数据难度：低到中。

可以构造大多数窗口都相似、少数窗口异常的序列：

```
normal window: sin wave + small noise
anomaly window 1: sudden spike
anomaly window 2: level shift
anomaly window 3: flatline
anomaly window 4: high variance segment
```

LOF 不直接处理原始点，而是处理窗口特征，因此还需要设计特征，例如均值、方差、最大值、最小值、斜率、峰值或变化幅度。

结论：数据可用性很好，测试数据也能构造，但特征设计和参数解释比变化点检测更复杂。

6.7 数据可行性排序

只从“是否有合适数据、是否方便构造测试数据”的角度看，排序如下：

1. 滑动窗口统计量 + CUSUM 变化点检测：公开数据多，人工数据极易构造，标签位置清楚。
2. Holt-Winters 指数平滑预测：预测数据多，季节性测试数据容易构造，评价指标标准。
3. 滑动窗口线性回归短期预测：测试数据最容易构造，但真实实验亮点较弱。
4. LOF 窗口异常检测：异常检测数据多，但需要额外设计窗口特征。
5. PAA 特征 + KMeans 聚类：可以用 UCR 或日曲线数据，构造难度可控。
6. DTW-KMeans 时间序列片段聚类：可用数据不少，但构造和计算都更麻烦。

综合来看，如果只考虑数据准备和实验落地，变化点检测和 Holt-Winters 预测最稳；如果想做聚类，PAA + KMeans 比 DTW-KMeans 更容易完成。

参考数据源：

- Monash Time Series Forecasting Repository: <https://forecastingdata.org/>
- UCI Individual Household Electric Power Consumption: <https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption>
- UCI Traffic Flow Forecasting: <https://archive.ics.uci.edu/dataset/734/traffic+flow+forecasting>
- UCR Time Series Classification Archive: <https://www.timeseriesclassification.com/>
- UCR Time Series Anomaly Archive: https://figshare.com/articles/dataset/UCR_Time_Series_Anomaly_Detection_datasets_2021_/26410744
- Numenta Anomaly Benchmark: <https://github.com/numenta/NAB>

7. IoTDB 已有实现与重复风险查询

查询 IoTDB 官方 UDF Library 文档和 IoTDB 2.0 AInode 文档后，六个候选选题的已有实现情况如下。

选题	IoTDB 是否已有相近实现	重复风险	说明
Holt-Winters 指数平滑预测	AINode 中已有 <code>holtwinters</code> 内置模型；UDF Library 中未查到同名 UDF	中到高	如果课程要求必须做 UDF，仍可实现 UDF 版本；但如果老师认为 AINode 内置模型也算 IoTDB 已实现，则重复风险较高。
滑动窗口线性回归短期预测	UDF Library 中有 <code>segment</code> 线性趋势分段、 <code>AR</code> 自回归系数学习，但未查到直接的滚动线性预测 UDF	中	不是完全重复，但和已有线性拟合、AR 学习功能有一定相邻关系。
DTW-KMeans 时间序列片段聚类	UDF Library 中已有 <code>DTW</code> 距离函数和 <code>Cluster</code> 聚类函数	中到高	没查到“DTW-KMeans”完整实现，但 DTW 和聚类两个核心组件都已有，选题需要强调“基于 DTW 距离的窗口聚类”与官方 <code>Cluster</code> 的差异。
PAA 特征 + KMeans 聚类	UDF Library 中已有 <code>Cluster</code> 聚类函数；未查到 <code>PAA</code> 同名函数	中	KMeans 窗口聚类已有相近实现。如果选择该方向，应突出 PAA 降维、特征输出、日曲线聚类解释等差异。
滑动窗口统计量 + CUSUM 变化点检测	未查到 <code>CUSUM</code> 、 <code>ChangePoint</code> 同名 UDF；但已有 <code>IQR</code> 、 <code>KSigma</code> 、 <code>LOF</code> 、 <code>Range</code> 等异常检测 UDF	低到中	异常检测大类已有很多函数，但“变化点检测 + CUSUM”没有看到直接实现，重复风险最低。
LOF 窗口异常检测	UDF Library 已有 <code>LOF</code> ，注册类为 <code>org.apache.iotdb.library.anomaly.UDTFLOF</code>	高	该方向与官方实现高度重合，不建议作为最终选题，除非做明显扩展，例如解释性特征、窗口形态异常、与现有 LOF 的对比增强。

参考依据：

- IoTDB UDF Library 文档中已有 `LOF`、`DTW`、`Cluster`、`AR`、`Segment` 等函数。
- IoTDB AINode 文档中 `SHOW MODELS` 示例列出了 `holtwinters`、`arima`、`exponential_smoothing`、`stray` 等内置模型。
- 在 UDF Library 文档中未查到 `Holt`、`PAA`、`CUSUM`、`ChangePoint` 等同名 UDF。

从“避免和 IoTDB 已有实现重复”的角度，建议重新看待六个选题：

- 最推荐：滑动窗口统计量 + CUSUM 变化点检测。没有查到直接同名实现，且仍属于异常检测，数据和实验都好做。
- 可以考虑：PAA 特征 + KMeans 聚类。官方已有 `Cluster`，但如果强调 PAA 特征工程和聚类解释，仍有差异空间。
- 可以考虑但需说明差异：DTW-KMeans 聚类。官方已有 `DTW` 和 `Cluster`，需要把差异写清楚。
- 谨慎选择：Holt-Winters 预测。AINode 已有 `holtwinters` 模型，做 UDF 版本需要说明课程目标是 UDF 封装和轻量实现。
- 谨慎选择：滑动窗口线性回归预测。重复风险不算最高，但算法亮点弱。
- 不建议：LOF 窗口异常检测。官方 UDF 已经直接实现 LOF，重复风险最高。

因此，如果老师比较在意“不重复已有函数”，当前最稳的选题仍然是：

基于滑动窗口统计量与 CUSUM 的 IoTDB 时间序列变化点检测 UDF 实现。

8. 三个方向横向比较

方向	推荐选题	实现难度	展示效果	报告可写性	风险
预测	Holt-Winters 预测	中	中	强	未来时间戳输出和参数选择
预测	滑动窗口线性回归预测	低	中	中	算法亮点偏弱
分类/聚类	DTW-KMeans 聚类	高	强	强	计算量和聚类评价
分类/聚类	PAA + KMeans 聚类	中	中	中	算法特色略弱
异常检测	滑动窗口统计量 + CUSUM 变化点检测	低到中	强	强	阈值选择
异常检测	LOF 窗口异常检测	中到高	强	强	复杂度和特征设计

9. 综合评分与排序

评分采用 100 分制，主要考虑四个维度：

- 实现可控性：35 分。实现越稳定、越不依赖复杂工程细节，分数越高。
- 展示效果：25 分。实验图、结果解释和可视化越直观，分数越高。
- 报告可写性：25 分。算法背景、问题定义、输入输出和评价指标越清楚，分数越高。
- 选题亮点：15 分。算法代表性、数据挖掘感和差异化越强，分数越高。

排名	方向	选题	实现可控性	展示效果	报告可写性	选题亮点	总分
1	异常检测	滑动窗口统计量 + CUSUM 变化点检测	31	23	23	12	89
2	预测	Holt-Winters 指数平滑预测	27	20	24	12	83
3	异常检测	LOF 窗口异常检测	22	23	22	14	81
4	分类/聚类	DTW-KMeans 时间序列片段聚类	18	23	23	15	79
5	分类/聚类	PAA 特征 + KMeans 聚类	28	19	20	11	78
6	预测	滑动窗口线性回归短期预测	33	16	18	7	74

按总分重新排序后，建议为：

1. 基于滑动窗口统计量与 CUSUM 的 IoTDB 时间序列变化点检测 UDF：89 分。
2. 基于 Holt-Winters 指数平滑的 IoTDB 时间序列预测 UDF：83 分。
3. 基于 LOF 的 IoTDB 时间序列异常检测 UDF：81 分。
4. 基于 DTW-KMeans 的 IoTDB 时间序列片段聚类 UDF：79 分。
5. 基于 PAA 特征与 KMeans 的 IoTDB 时间序列聚类 UDF：78 分。
6. 基于滑动窗口线性回归的 IoTDB 短期趋势预测 UDF：74 分。

上面的分数主要评价实现难度、展示效果和报告可写性，没有把“IoTDB 是否已有相近实现”作为扣分项。结合第 7 节的查询结果，如果把重复风险也考虑进去，最终建议排序调整为：

1. 基于滑动窗口统计量与 CUSUM 的 IoTDB 时间序列变化点检测 UDF。
2. 基于 PAA 特征与 KMeans 的 IoTDB 时间序列聚类 UDF。
3. 基于 DTW-KMeans 的 IoTDB 时间序列片段聚类 UDF。
4. 基于 Holt-Winters 指数平滑的 IoTDB 时间序列预测 UDF。
5. 基于滑动窗口线性回归的 IoTDB 短期趋势预测 UDF。
6. 基于 LOF 的 IoTDB 时间序列异常检测 UDF。

最推荐的是第 1 个变化点检测方向。它属于异常检测，算法实现可控，UDF 输出形式简单，实验图也最容易做出效果。

如果希望选题更偏预测，可以优先考虑 Holt-Winters。它比线性回归更有算法完整性，也更适合写正式报告。

如果希望选题更偏分类/聚类，DTW-KMeans 的亮点更强，但实现风险也更高；PAA + KMeans 更稳。如果课程更看重完成质量，建议选 PAA + KMeans；如果小组愿意承担更多实现压力，可以选 DTW-KMeans。

10. 后续推进建议

确定最终方向后，建议按以下步骤推进：

1. 明确最终题目和 1 到 2 个核心 UDF 函数名。
2. 写清楚输入、参数、输出类型和 SQL 调用示例。
3. 先实现不依赖 IoTDB 的算法核心类。
4. 用人工构造数据做单元测试。
5. 再封装 IoTDB UDF。
6. 选择一个公开数据集做真实实验。
7. 输出结果图，完成说明文档和展示 PPT。

为了控制工作量，第一版不建议同时做多个方向。最终报告中只需要把一个方向讲完整，比把预测、聚类和异常检测都做一点更稳妥。